API serverless observable en Google Cloud

Andy
30 de octubre de 2026

Desplegué una API de pedidos, generé una hora de tráfico y analicé sus métricas y logs. Completé **6000 solicitudes** sin errores de conexión. El escenario elevó los errores 4xx de electronics de **8.82 % a 15.10 %**.

Implementación y metodología

Implementé crear, consultar y cancelar pedidos en Cloud Run functions con Python 3.13 y entrada **handle**. Desplegué mediante Terraform en un proyecto de Google Cloud, región us-central1. Conservé el estado en Firestore orders-lab; la cancelación utiliza una transacción y rechaza cambios de estado imposibles.

Parámetro	Valor utilizado
Distribución de envíos	Normal N(30, 10) en minutos, truncada a [0, 60]
Solicitudes y semilla	6000 solicitudes; seed = 42
Errores intencionales	invalid_share = 0.08
Fallas y demora	FAILURE_RATE = 0.02; EXTRA_LATENCY_MS = 0
Escenario	electronics; minutos 35 a 45; double_errors
Fecha de la corrida	30 de octubre de 2026
Duración medida	3600.005 segundos

Antes de iniciar preparé 18 solicitudes de control y creé la métrica basada en logs. La preparación quedó separada de las 6000 solicitudes medidas. Envié X-Request-ID y emití logs JSON con resultado, error, latencia y atributos de negocio.

Resultados generales

Respuesta	Cantidad	Porcentaje
Éxitos 200 y 201	5381	89.68 %
Errores 400, 404 y 409	497	8.28 %
Errores 500	122	2.03 %
Total	6000	100.00 %

Validé 6000 filas, 6000 IDs únicos y cero errores de transporte en el CSV. Los porcentajes mostrados están redondeados de forma independiente.

Tráfico durante el experimento

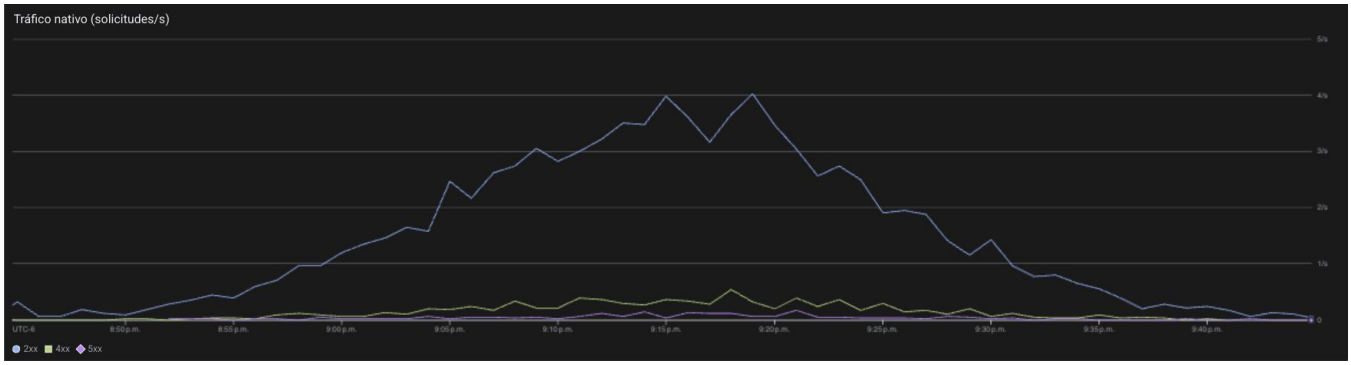


Figura 1. Tráfico nativo de Cloud Run separado por clase de respuesta. Captura real de GCP.

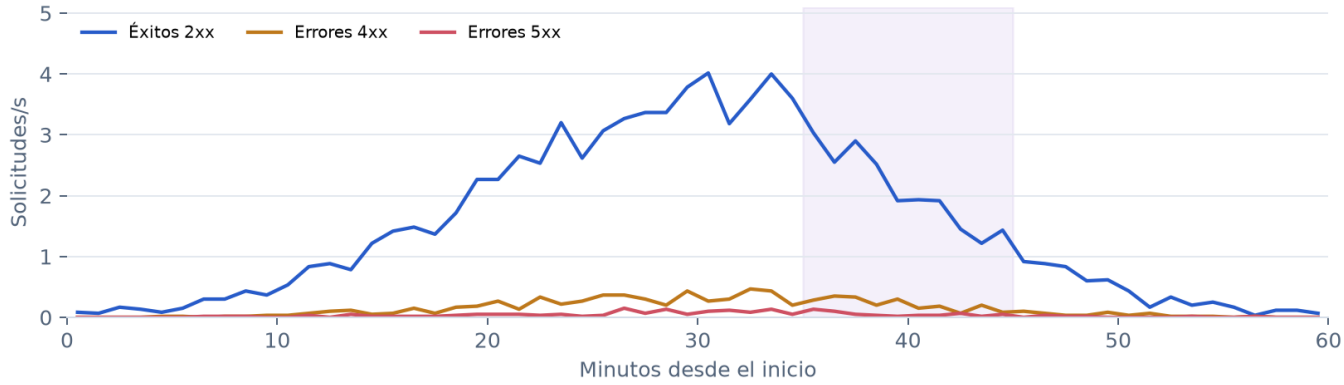


Figura 2. Tráfico reconstruido desde requests.csv; cada punto resume un minuto desde el inicio. La franja marca el escenario de electronics.

Observé un aumento gradual del tráfico y una disminución posterior, coherentes con la normal configurada. En el CSV, el minuto 33 tuvo **274 solicitudes**, equivalentes a **4.57 solicitudes/s**. La tasa media de la hora fue 1.67 solicitudes/s. El pico de éxitos 2xx del panel nativo es menor que el pico total porque separa los códigos de respuesta.

Relación entre tráfico y latencia

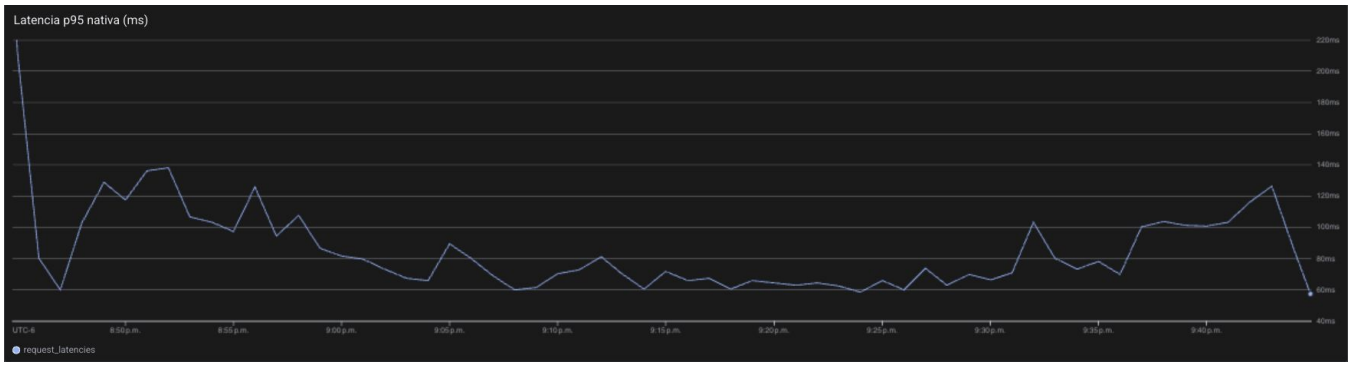


Figura 3. Percentil 95 de latencia nativa de Cloud Run en la hora completa.

El p95 nativo osciló aproximadamente entre **50 y 212 ms**. No observé una subida sostenida coincidente con el pico de tráfico. Esto describe esta corrida; no demuestra que la carga nunca afecte la latencia ni permite atribuir los picos a una causa específica.

Latencia por operación y observabilidad

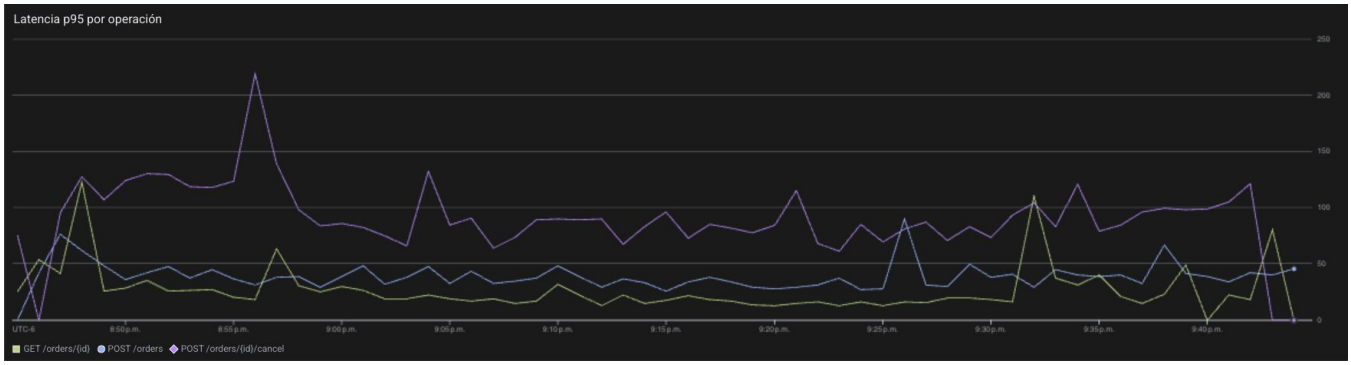


Figura 4. Consulta SQL sobre latency_ms de los logs de aplicación, agrupada por minuto y endpoint.

Cancelar pedidos mostró mayor p95 de aplicación durante buena parte del periodo, con un pico cercano a **219 ms**. La transacción y el trabajo adicional en Firestore son una explicación posible; no realicé un experimento separado que confirme esa causa.

Medición del cliente	Resultado
Latencia media	144.82 ms
Mediana	131.71 ms
p95 global	236.93 ms
Máximo individual	4350.61 ms

La latencia del cliente incluye red y respuesta HTTP; el log mide el tiempo dentro del manejador y Cloud Run mide la solicitud en la plataforma. No utilicé sus percentiles como valores intercambiables ni promedié p95 por minuto para calcular un p95 global.

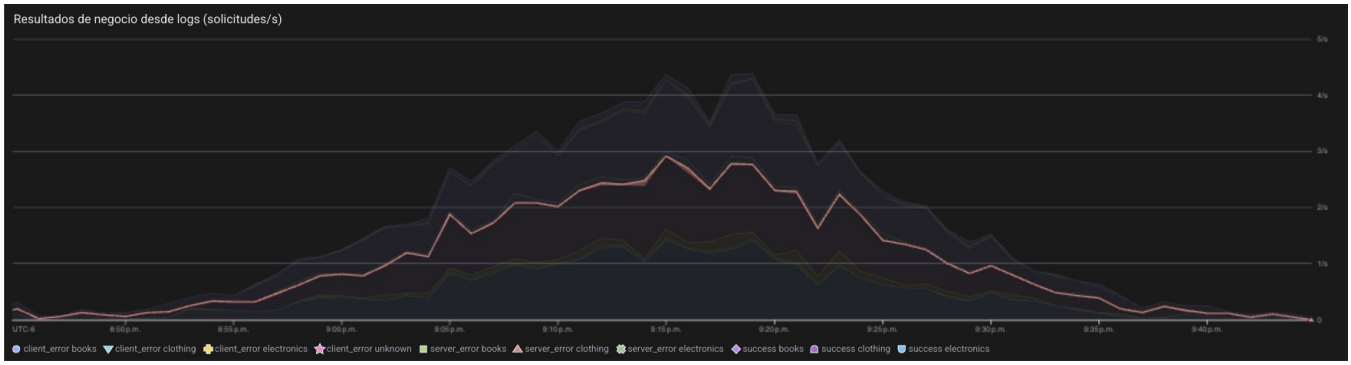


Figura 5. Métrica basada en logs por outcome y category, creada antes de la corrida.

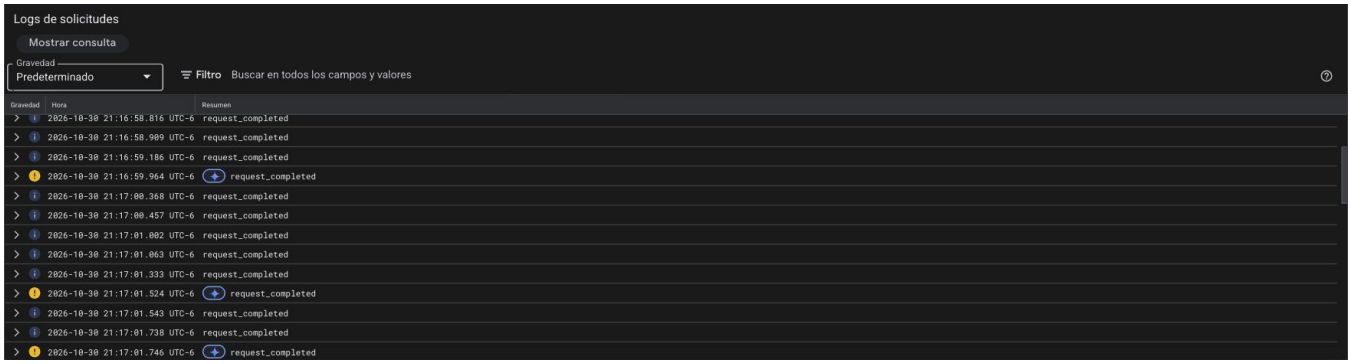


Figura 6. Panel de logs de solicitudes dentro del mismo dashboard. Los registros conservan el request_id para correlacionarlos con el CSV.

Efecto del escenario en electronics

Entre los minutos **35 y 45**, dupliqué la probabilidad de eventos inválidos de electronics de 8 % a 16 %. La probabilidad de fallas internas permaneció en 2 %. Comparé tasas para considerar las diferencias de volumen.

Categoría	Minutos	Envíos	4xx	Tasa 4xx
electronics	Fuera	1531	135	8.82 %
electronics	35 a 45	457	69	15.10 %
books	Fuera	1527	120	7.86 %
books	35 a 45	480	28	5.83 %
clothing	Fuera	1529	113	7.39 %
clothing	35 a 45	476	32	6.72 %

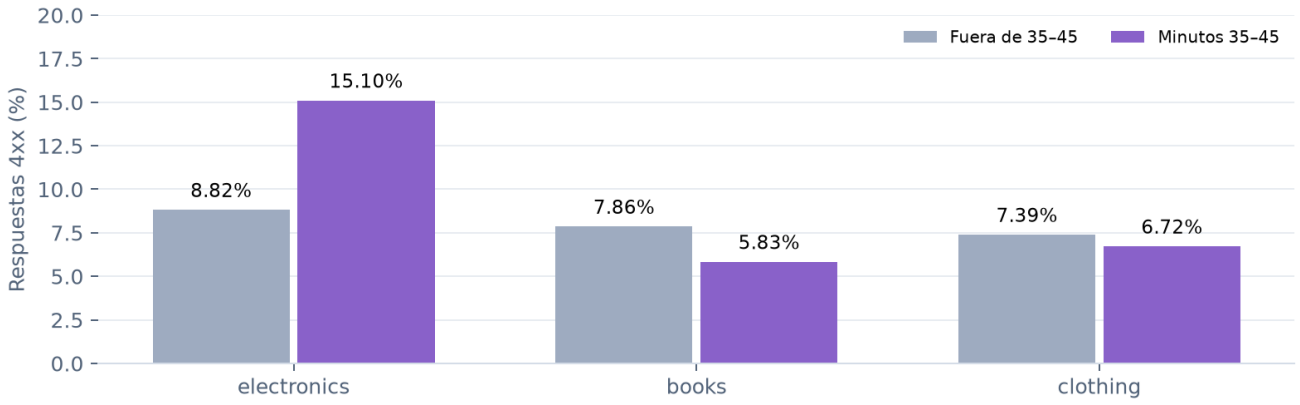


Figura 7. Comparación de tasas 4xx calculada desde las filas de la corrida.

La tasa 4xx de electronics pasó de **8.82 % a 15.10 %**: aumentó 6.28 puntos porcentuales, aproximadamente 1.71 veces. Sus errores totales, al sumar 4xx y 5xx, pasaron de 10.65 % a 16.85 %. Books y clothing no mostraron el mismo incremento agregado durante la ventana.

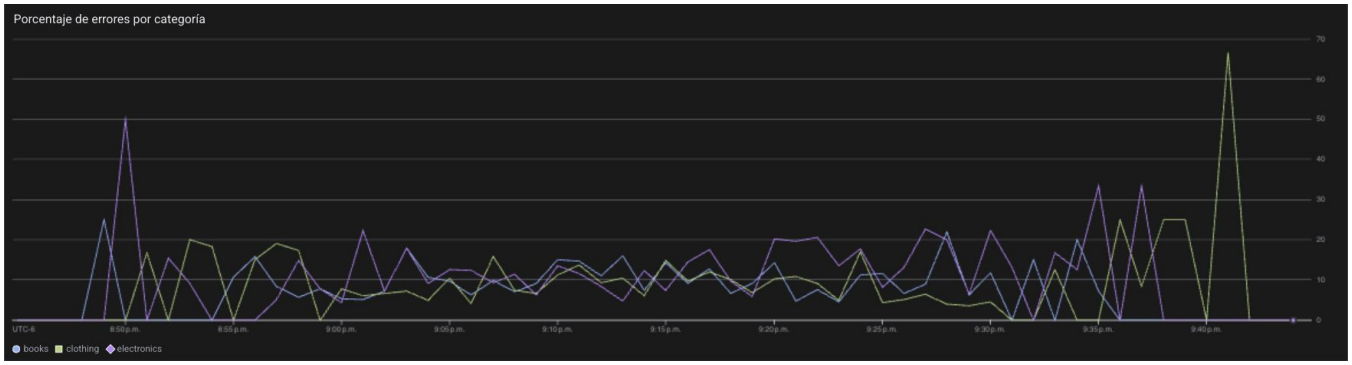


Figura 8. Porcentaje de errores por categoría obtenido con SQL en Log Analytics.

El aumento observado es consistente con el escenario. La tasa no tiene que duplicarse exactamente por la aleatoriedad y la mezcla de errores. Los picos al final de la gráfica corresponden a intervalos de pocos envíos y no deben interpretarse sin revisar su denominador.

Conclusiones y evidencias de entrega

Comprobé que la API acepta solicitudes válidas y distingue errores de payload, recursos inexistentes, conflictos de estado y fallas internas. El tráfico se concentró cerca de la media configurada y el escenario elevó la proporción de errores de electronics. Utilicé métricas de plataforma, logs estructurados y SQL para analizar el comportamiento en un mismo dashboard.

Dashboard del laboratorio



Figura 9. Dashboard único de Cloud Monitoring con sus seis paneles. Las figuras anteriores presentan cada panel con mayor detalle.

Conservé los paneles nativos de tráfico y p95, la métrica basada en logs por resultado y categoría, el panel de logs y las dos gráficas SQL de errores y latencia por operación. El dashboard HTML adjunto complementa estas evidencias con filtros sobre el CSV y funciona sin conexión.

Alcance de los resultados

El **10.32 %** de respuestas de error incluye eventos provocados deliberadamente; no lo presento como una medida de disponibilidad de producción. Los totales exactos corresponden al CSV de la corrida. Las métricas nativas y basadas en logs pueden incluir otras llamadas en el mismo rango; las consultas SQL filtran el prefijo del run_id.

No exporté las series numéricas nativas de GCP para calcular correlaciones o causas. Interpreto las capturas como evidencia visual y calculo los resultados exactos desde los archivos del simulador. La limpieza con terraform destroy queda para después de conservar y entregar las evidencias.

Pasos para reproducir

Crear un proyecto propio y habilitar facturación. Completar las plantillas locales, desplegar con Terraform y configurar la URL de la función. Crear las métricas antes de ejecutar el simulador durante una hora. Agregar las gráficas SQL al dashboard, guardar evidencias y eliminar los recursos al terminar.

Para la publicación conservé las mediciones originales y reemplacé las referencias de solicitudes por valores anónimos. Excluí identificadores de infraestructura, enlaces privados y archivos de autenticación.